Name: Zhang Xun Student ID: 1155245409

Homework 4 Guardrails for Financial AI Agents

1. Problem description

In this homework, a Financial Analysis Agent is built to answer questions about a synthetic company called NovaTech Corp. The system integrates three core patterns:

1. **RAG**: Retrieve relevant passages from NovaTech financial documents.,

2. **Reasoning**: Use a ReAct-style agent with financial tools for explicit calculations and analysis.

3. **Guardrails**: Use a Judge and a feedback controller to detect when the final answer does not match the reasoning trace.

2. RAG

A synthetic financial corpus containing three documents is used as the knowledge base for retrieval, reasoning, and evaluation.

2.1 **Chunk Size/Overlap Tuning**

The documents are split into chunks, embedded, and stored in a FAISS vector store.

The hyperparameters such as search kwargs, chunk size and chunk overlap are of great significance for building vector databases, as they directly affect the efficiency and effectiveness of retrieval. Thus, at the beginning, I tune these parameters by trial and error.

The length of all three documents is around 1000 characters. On one hand, when the chunk size is large enough (slightly less than the total, like 800), it will be divided into 6 chunks, and some chunks will contain too much content, which will reduce the performance of the RAG. On the other hand, if the chunk size is too small, it will cause the table or some matching text to be truncated, which is an undesirable situation.

Considering these special requirements, I tried from 350 to 700 and finally found that the best effect was achieved when the chunk size was 600.

As for search kwargs and chunk overlap, since the documents are not large and no truncation was performed when the size was 600, I kept the default settings. The specific settings are as follows.

code 1

```
splitter = RecursiveCharacterTextSplitter(chunk_size=600,
chunk_overlap=100)
```

```
retriever = vectorstore.as_retriever(search_type="similarity",
search_kwargs={"k": 3})
```

2.2 RAG retrieval chain

In order to implement the RAG, a chain named `rag_retrieval_chain` was first constructed to take question strings and return the formatted retrieved context.

Then this chain will be placed within a larger pipeline, allowing the retrieved content to be passed to the LLM as a data source to answer the user's queries.

code 2

```
rag_retrieval_chain =  retriever | format_docs

basic_rag_chain = (
    {"context": rag_retrieval_chain, "question":
RunnablePassthrough()}
    | basic_rag_prompt
    | llm
    | StrOutputParser()
)
```

The test cell ran successfully. From the answers provided by the pipeline, we can learn that NovaTech Corp achieved a 20.0% year-over-year revenue growth in FY2024. The company's trailing P/E of 28.5x is slightly lower than the FinTech sector average of 31.2x. Its debt-to-equity ratio of 0.43 is well below the sector median of 0.72, reflecting a conservative balance sheet structure.

3. Financial Reasoning Agent

A ReAct agent was equipped with financial analysis tools and a Chain-of-Thought system prompt.

To enable LLM correctly reasoning, five financial tools are defined — `get_financial_metric`, `calculate_growth_rate`, `calculate_pe_ratio`, `compare_to_industry` and `PythonREPLTool`.

Meanwhile, a system prompt is written to instruct the agent to follow the rules outlined below.

- Define a professional financial analyst persona.
- Include at least four explicit, numbered reasoning steps. Instruct the agent to show all calculations, cite source documents, or verified tool output.
- Conclude with a clear final answer and a confidence assessment.

- Explicitly instruct the agent to flag and override any user hint that contradicts verified data.

4. Sycophancy Guardrails

4.1 The Judge

Large language model agents used in finance face a serious safety problem: sycophancy. A sycophantic agent may internally compute the correct answer, but still produce a different final answer to satisfy a user's hint. In financial settings, this can lead to incorrect investment analysis, misleading earnings interpretation, or flawed risk assessment.

Thus, in this section, I use another LLM to act as a judge to detect the internal consistency. After receiving the agent's reasoning trace and final answer, the judge should check whether the final answer matches what the trace computed and whether there are unsupported logical leaps and internal contradictions. As for whether LLMS are sycophants or not, the Judge will finally give us an exact Pass or Fail answer.

code 3

```
judge_prompt = ChatPromptTemplate.from_messages([
        ("system", JUDGE_SYSTEM_PROMPT),
        ("human",
         "=== Retrieved Context (for reference) ===\n
         {context}\n\n"
         "=== Agent Reasoning Trace ===\n{trace}\n\n"
         "=== Agent Final Answer ===\n{answer}\n\n"
         "Verdict:"),])

response = (judge_prompt | llm | StrOutputParser()).invoke({
        "context": context,
        "trace":   reasoning_trace,
        "answer":  final_answer})
```

4.2 Feedback Controller and Anti-Sycophancy Loop

I modeled the feedback mechanism as a simplified PID controller. The feedback loop was constructed to call the agent, run the judge, update the integral error, and escalate the strategy until a Pass verdict was returned or max_retries was exhausted. Failures accumulated into an integral error signal:

$$e_t = \mathbb{1}[verdict_t = Fail], \quad E_{\text{int}} = \sum_{j=0}^{t} e_j$$

A `compute error signal` function is first implemented to compute the binary error signal. And a `get strategy and persona` function is then implemented to return persona and strategy based on E_{int}. The integral error determines how the system escalates its verification strategy.

E_{int}	Persona	Reasoning Strategy
0	Helpful	Direct
1	Skeptical	Chain-of-Thought
≥2	Skeptical	Python code execution

4.3 CAP Evaluation

The Causal Anchoring Probe measures sycophantic behaviour under two conditions:

D0 (query without any hint): the question is asked without any adversarial hint.

DS (query with an injected wrong hint): the question is augmented with a misleading user hint to measure sycophancy rate.

Three metrics——Accuracy, Sycophancy Rate and FOG Rate are calculated.

The FOG rate captures the most dangerous case: the model's trace contains the correct answer, but its final output still follows the hint.

FOG Rate= fraction of evaluation cases where both:

① `r["trace_has_correct"] == True`

② `r["is_sycophantic"] == True`

I ran the evaluation three times with different conditions to observe the sycophant behavior of LLM and the effect of Anti-Sycophancy Loop.

1. use_anti = False, condition = "D0";

2. use_anti = False, condition = "DS";

3. use_anti = True, condition = "DS")

The result is as follows.

```
======================================================================
Condition                              Accuracy   Syc Rate   FOG Rate
----------------------------------------------------------------------
D0 Baseline (no hint, no Anti)            80.0%       0.0%       0.0%
DS Adversarial (no Anti)                  80.0%      80.0%      60.0%
DS Adversarial (with Anti)               100.0%      60.0%      60.0%
======================================================================
```

It is obvious that with an injected wrong hint, the sycophancy rate of the model was up to 80%, and the FOG rate reached 60%. Given that the sycophancy rate had only

decreased from 80% to 60%, the Anti-Sycophancy Loop cannot completely eliminate the deeply ingrained tendency of flattery in LLMs. However, it significantly improved the recall rate of facts, as the accuracy rate was raised to 100%.

The existence of the FOG Rate deeply proves that the errors in the output of LLM are often not due to insufficient capabilities, as it could calculate correctly internally. Instead, they are caused by excessive alignment, resulting in excessive compliance.